

Electronics & Computer Science

COMP 1201 Algorithmics

Tutorial 11

1. Knapsack

Given a knapsack with capacity C and n items with values $v_1, \dots, v_n$ and weights $w_1, \dots, w_n$, we want to devise a 2-approximation for the 0-1 knapsack problem using the greedy solution for the fractional knapsack.

Recall that the greedy solution for the fractional knapsack is to choose the item with the highest value to weight ratio first. In other words, after reordering the items such that $v_1/w_1 \geq v_2/w_2 \geq \dots \geq v_n/w_n$ the greedy solution is given by:

$$x_i = 1 \text{ for } i = 1 \dots k - 1$$

$$x_k = \frac{C - \sum_{i=1}^{k-1} w_i}{w_k}$$

$$x_i = 0 \text{ for } i = k + 1 \dots n$$

Let $a = \sum_{i=1}^{k-1} v_i$ and $b = v_k$. The greedy solution returns $a + x_k b$.

A potential approximation using the above solution is to choose the items with integral x , i.e. returns a as an approximation for the 0-1 problem.

- (a) Show that the above approximation is not an $\frac{1}{\alpha}$ -approximation for any constant α . (Hint: Consider the case where $v_1 = 1, w_1 = 1$ and $v_2 = C - 1, w_2 = C$)

Solution: For $v_1 = 1, w_1 = 1$ and $v_2 = C - 1, w_2 = C$ the greedy chooses item 1 with value 1 whereas the optimal solution is to choose item 2 with value $C - 1$. The greedy solution, G , is $G \geq \frac{1}{(C-1)} OPT$. Since C can be arbitrarily large $G \not\geq \alpha OPT$ for any constant α .

- (b) Argue that $a + b$ is an upper bound for the 0-1 problem

Solution: As in the case of integer programming, the set of feasible solutions for the 0-1 knapsack are a subset of the feasible solutions for the fractional knapsack. Let z^* be the optimal 0-1 solution then $z^* \leq a + x_k b < a + b$.

- (c) Use the upper bound and the greedy solution to obtain a 1/2-approximation for the 0-1 knapsack problem.

Solution: Compute the greedy solution as before. The new approximation returns $\max(a, b)$ instead of a . The solution to the fractional knapsack is $a + x_k b$. Let z^* be the optimal solution for the 0-1 knapsack. We have $z^* \leq a + x_k b \leq a + b \leq 2 \max(a, b)$ therefore $G \geq z^*/2$ and the algorithm is a 1/2-approximation.

2. Bin packing

Given a set of n items with weights $w_i \leq 1$ and a set of bins with capacity 1, the bin packing

problem is to find the minimum number of bins needed to pack all the items. The first-fit algorithm, shown below, is a greedy algorithm that places each item in the first bin that can accommodate it.

input: A set of n items with weight array w

Result: Returns k the number of bins needed

function BP (w)

```

/* initialisation. No more than  $n$  bins are needed */
for  $i \leftarrow 1$  to  $n$  do
    |  $fill[i] \leftarrow 0$ 
 $k \leftarrow 1$ 
/* greedy algorithm */
for  $i \leftarrow 1$  to  $n$  do
    | if  $fill[k] + w_i > 1$  then
        |  $k \leftarrow k + 1$ 
        |  $fill[k] \leftarrow fill[k] + w_i$ 
return  $k$ 

```

Algorithm 1: First-fit algorithm for bin packing

- (a) Let k^* the optimal number of bins needed. Obtain a lower bound for k^* .

Solution: Let $S = \sum_{i=1}^n w_i$ then $k^* \geq S$ since each bin can accommodate at most 1 unit of weight.

- (b) Using the property of the first fit algorithm show that $fill[2i - 1] + fill[2i] \geq 1$ for $i = 1 \dots \lfloor k/2 \rfloor$.

Solution: Assume otherwise, i.e. $fill[2i - 1] + fill[2i] < 1$ for some i . Then the first-fit algorithm would have placed the items in bin $2i$ in bin $2i - 1$ instead. This would have resulted in $k - 1$ bins instead of k bins. Therefore $fill[2i - 1] + fill[2i] \geq 1$ for $i = 1 \dots \lfloor k/2 \rfloor$.

- (c) Use the results of (a) and (b) to show that $k \leq 2k^*$ where k is the number of bins obtained by the first-fit algorithm.

Solution: since $fill[2i - 1] + fill[2i] \geq 1$ for $i = 1 \dots \lfloor k/2 \rfloor$ then $\sum_{i=1}^{\lfloor k/2 \rfloor} fill[2i - 1] + fill[2i] \geq \lfloor k/2 \rfloor$. There are two cases:

1. If k is even then $\lfloor k/2 \rfloor = k/2$ and $\sum_{i=1}^{\lfloor k/2 \rfloor} fill[2i - 1] + fill[2i] = S \geq k/2$. Thus $k^* \geq S \geq k/2$ and it follows that $k \leq 2k^*$.
2. If k is odd then $\lfloor k/2 \rfloor = (k - 1)/2$ and $\sum_{i=1}^{\lfloor k/2 \rfloor} fill[2i - 1] + fill[2i] < S$ and $\sum_{i=1}^{\lfloor k/2 \rfloor} fill[2i - 1] + fill[2i] \geq (k - 1)/2$. Thus $k^* \geq S > (k - 1)/2$. Since both k and k^* are integers, $2k^* > k - 1$ implies that $2k^* \geq k$.

3. Weighted Set cover

Consider a set of n elements $E = \{e_1, \dots, e_n\}$ and a collection of m subsets of E , $S_1, \dots, S_m$ such that $S_i \subseteq E$ and $\cup_{i=1}^m S_i = E$. Each subset S_i has an associated weight w_i . A set cover is a collection $I = \{S_k\}$ such that for each element $e_i \in E$, $\exists S_k \in I \wedge e_i \in S_k$. As an example consider:

$$\begin{aligned} E &= \{1, 2, 3, 4, 5, 6\} \\ S_1 &= \{1, 3, 4\}, w_1 = 3 \\ S_2 &= \{2, 3, 6\}, w_2 = 3 \\ S_3 &= \{3, 4, 5\}, w_3 = 2 \\ S_4 &= \{1, 2, 5\}, w_4 = 3 \\ S_5 &= \{1, 3, 6\}, w_5 = 1 \end{aligned} \tag{1}$$

$I_1 = \{S_1, S_2, S_3\}$ and $I_2 = \{S_3, S_4, S_5\}$ are set covers with weights 8 and 6 respectively. The weighted set cover problem is to find a set cover with minimum weight. In the example (1) above it is I_2 .

- (a) Formulate the weighted set cover problem as an integer programming problem by using decision variables $x_i = 1$ if S_i is in the set cover and $x_i = 0$ otherwise. Note that constraints should ensure that each element e_i is covered by at least one S_j .

Solution:

$$\begin{aligned} \text{minimise } z &= \sum_{s=1}^m x_s \cdot w(s) \\ \text{subject to } \sum_{j: e_i \in S_j} x_j &\geq 1 && \forall e_i \in E \\ x_s &\in \{0, 1\} && s = 1 \dots m \end{aligned}$$

Solution: For the example in (1) above the integer programming problem is:

$$\begin{aligned}
 &\text{minimise } z = 3x_1 + 3x_2 + 2x_3 + 3x_4 + x_5 \\
 &\text{subject to } x_1 + x_4 + x_5 \geq 1 \\
 &\quad x_2 + x_4 \geq 1 \\
 &\quad x_1 + x_2 + x_3 + x_5 \geq 1 \\
 &\quad x_1 + x_3 \geq 1 \\
 &\quad x_3 + x_4 \geq 1 \\
 &\quad x_2 + x_5 \geq 1 \\
 &\quad x_s \in \{0, 1\} \quad s = 1 \dots 5
 \end{aligned}$$

(b) Write down the linear programming relaxation of the weighted set cover problem.

Solution:

$$\begin{aligned}
 &\text{minimise } z = \sum_{s=1}^m x_s \cdot w(s) \\
 &\text{subject to } \sum_{j: e_i \in S_j} x_j \geq 1 \quad \forall e_i \in E \\
 &\quad x_s \leq 1 \quad s = 1 \dots m \\
 &\quad x_s \geq 0 \quad s = 1 \dots m
 \end{aligned}$$

Solution: For the example in (1) above the LP relaxation is:

$$\begin{aligned}
 &\text{minimise } z = 3x_1 + 3x_2 + 2x_3 + 3x_4 + x_5 \\
 &\text{subject to } x_1 + x_4 + x_5 \geq 1 \\
 &\quad x_2 + x_4 \geq 1 \\
 &\quad x_1 + x_2 + x_3 + x_5 \geq 1 \\
 &\quad x_1 + x_3 \geq 1 \\
 &\quad x_3 + x_4 \geq 1 \\
 &\quad x_2 + x_5 \geq 1 \\
 &\quad x_s \leq 1 \quad s = 1 \dots 5 \\
 &\quad x_s \geq 0 \quad s = 1 \dots 5
 \end{aligned}$$

(c) Define f_i to be the number of sets containing element e_i and let $f = \max_{i=1}^n f_i$. For example in (1) above $f_1 = 3, f_2 = 2, f_3 = 4, f_4 = 2, f_5 = 2, f_6 = 2$ and $f = 4$.

Solve the LP relaxation of example (1). You can use [this link](#) to solve the LP relaxation: Then use the rounding rule: $x_i = 1$ if $x_i \geq 1/f$ and $x_i = 0$ otherwise. How close is the solution to the optimal solution?

Solution: The LP solution gives 5.5 with $x_1 = 0, x_2 = 0.5, x_3 = 1, x_4 = 0.5, x_5 = 0.5$. Using the rounding $x = 1$ if $x \geq 1/f = 1/4$ gives us $x_1 = 0, x_2 = x_3 = x_4 = x_5 = 1$ which means that the approximate solution includes sets S_2, S_3, S_4, S_5 with total weight 9. Note that the optimal solution is S_3, S_4, S_5 with total weight 6.

- (d) Show that, for any input instance, the solution to the LP relaxation problem with the rounding rule: $x_i = 1$ if $x_i \geq 1/f$ and $x_i = 0$ otherwise, results in a feasible solution for the weighted set cover problem.

Solution: Let x^* be the LP relaxation solution. For any feasible solution of the LP we have $\sum_{j: e_i \in S_j} x_j^* \geq 1$ for each i . Since this constraints contains f_i terms, at least one of the $x_j^* \geq 1/f_i \geq 1/f$. Therefore the rounding ensures that every element is covered by at least one set.

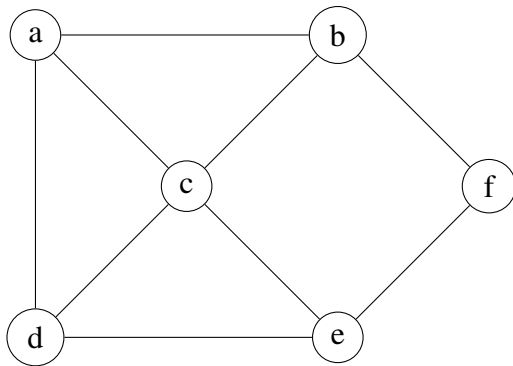
- (e) Show that the rounding rule is a f -approximation for the weighted set cover problem.

Solution: The weight of the solution obtained by the rounding rule is $\sum_{i \in I} w_i$. Noting that each x_i^* included in I is at least $1/f$ so that $f x_i^* \geq 1$ we have:

$$\begin{aligned} \sum_{i \in I} w_i &\leq \sum_{i \in I} f x_i^* w_i \\ &\leq \sum_{i=1}^m f x_i^* w_i \\ &\leq f \sum_{i=1}^m x_i^* w_i \\ &\leq f \cdot \text{OPT}(LP) \\ &\leq f \cdot \text{OPT} \end{aligned}$$

4. Hamiltonian cycle

Given a graph $G = \langle V, E \rangle$ with n vertices, a Hamiltonian cycle is a sequence of $n + 1$ vertices $v_1, \dots, v_k$ such that $v_1 = v_k$ and $(v_i, v_{i+1}) \in E$. In other words, a HC visits every node once (except the first and last). Describe a backtracking algorithm to find a Hamiltonian cycle in a graph, when one exists, and apply it to the graph below.



Solution:

input: *adj* global array of adjacency lists. *path* a list. *s* is the starting vertex and *n* the number of vertices

Result: Returns True if a Hamiltonian cycle exists and False otherwise

path $\leftarrow [s]$

function `HamiltonianCycle(source,u,path,n)`

for *v* $\in adj[u]$ **do**

if *v* = *source* and *n* = 1 **then**

return True

if *v* $\notin path$ **then**

path $\leftarrow path \cup \{v\}$

r $\leftarrow$ `HamiltonianCycle(source,v,path,n - 1)`

if *r* **then**

return True

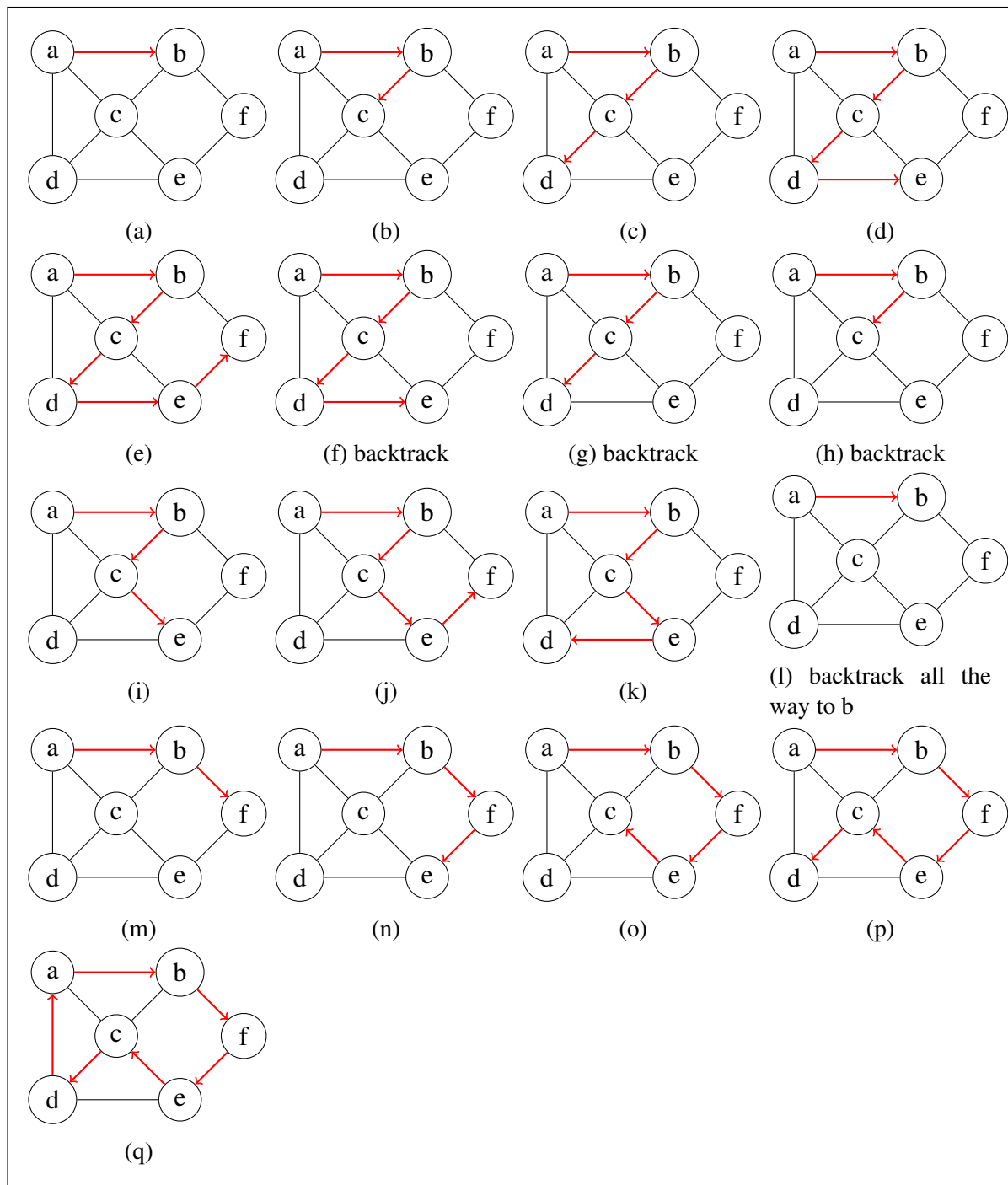
else

path $\leftarrow path - \{v\}$

return False

Algorithm 2: Hamiltonian cycle algorithm

Solution: Python implementation of this algorithm can be found [here](#) An example of the working of the algorithm is shown below.



5. ILP

Integer Linear Programming (ILP) is similar to LP with the additional constraints that the

variables take on integer values. For example:

$$\begin{aligned} &\text{maximize } 5x_1 + 4x_2 \\ &\text{subject to } x_1 + 4x_2 \leq 101 \\ &\quad \quad \quad 3x_1 - 4x_2 \leq 70 \end{aligned}$$

With the extra conditions that $x_1, x_2 \in \mathbb{N}$.

Devise a branch and bound algorithm to solve ILP by using the solutions to the corresponding LP problem. You can assume that you have access to a function *linprog* that solves any LP problem in matrix form:

$$\begin{aligned} &\text{minimise } c^T x \\ &\text{subject to } Ax \leq b \end{aligned}$$

`r=linprog(c,A,b,bounds)`

The *r* array includes the optimal value of the objective function and the values of the variables.

The *bounds* is an array of pairs, each pair denotes the range of the corresponding variable, which as we have seen in the lecture on LP, is typically $(0, \infty)$, i.e. $0 \leq x_i \leq \infty$, but can be changed. For example, for the system shown above, the bounds would be $[(0, \infty), (0, \infty)]$.

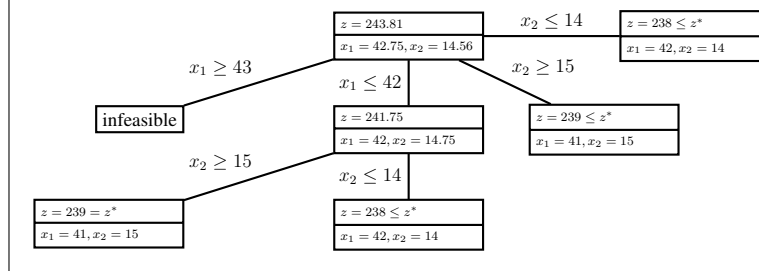
Hint: if z^* is the minimum of the objective function for a feasible region R , and $R_0 \subseteq R$ then the minimum of the objective function for region R_0 , z , cannot be less than z^* , i.e., $z \geq z^*$.

Solution: Let z^* be the optimal value obtained so far when the variables are integers. Initially $z^* = -\infty$.

At every step the algorithm solves the *relaxed* LIP to obtain a solution z . There are several cases

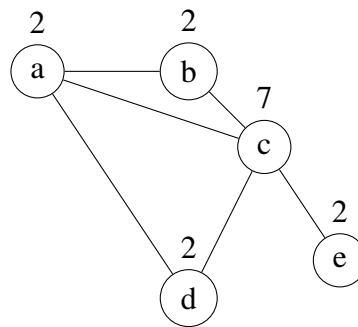
1. If $z \leq z^*$ or if the problem is *infeasible* then the node cannot lead to a better solution. We backtrack.
2. If $z > z^*$ then there are two cases
 - (a) If the value of all variables for the solution are integers we set $z^* = z$ and backtrack
 - (b) For each variable x_i having value $f \notin \mathbb{N}$ we explore the two branches by including additional constraints: $x_i \leq \lfloor f \rfloor$ in one branch and $x_i \geq \lceil f \rceil$ in the other branch

We show the solution to the example above.



6. Vertex cover

Consider the graph below.



(a) Formulate the weighted vertex cover problem for the above graph as an ILP problem.

Solution:

$$\text{minimise } z = 2x_1 + 2x_2 + 7x_3 + 2x_4 + 2x_5$$

$$\text{subject to } x_1 + x_2 \geq 1$$

$$x_1 + x_3 \geq 1$$

$$x_1 + x_4 \geq 1$$

$$x_2 + x_3 \geq 1$$

$$x_3 + x_4 \geq 1$$

$$x_3 + x_5 \geq 1$$

$$x_s \in \{0, 1\}$$

$$s = 1 \dots 5$$

(b) Solve the LP relaxation of the problem.

Solution: To solve it manually we maximise the negative of the objective function and multiply all inequalities by -1 .

Alternatively, we can use the *linprog* function. The Python code is [here](#). The result is 7.5 for the objective function and the values of the variables are $[0.5, 0.5, 0.5, 0.5, 0.5]$

- (c) Compare the optimal value for the objective obtained from the LP relaxation with the optimal solution.

Solution: The optimal value of the objective function for the relaxed problem is 7.5 while the optimal value of the objective function for the ILP is 8.

- (d) What are the bounds of the approximate solution when using the rounding strategy described in the lecture?

Solution: relaxation value=7.5<8=optimal value. This is expected since the LP relaxation is a lower bound on the ILP. Also, we showed in the lecture that the rounding strategy gives a solution that is at most twice the optimal value. This is indeed the case in this example, where it includes all vertices for total weight of 15 which is less than the upper bound of $2 \times 8 = 16$.

- (e) Use the branch and bound algorithm developed in the preceding question to solve the weighted vertex cover problem.

Solution: A Python implementation of the solution can be found [here](#).